

You can download the code for the above example at <http://goo.gl/sIVtDb>. Extract the zip file, open the project in NetBeans, add the API JAR files to the project as mentioned above and run the project.

The SMSLib API

A common requirement from clients is to integrate SMS functionality in their application. They may need it for sending order notifications, automated greetings, marketing messages, etc. Often this can be done using the bulk SMS gateways via the Internet. However, sometimes, using such gateways might not be a desired option. Clients may want to use their mobile or a GSM dongle to send such SMS. This method has two benefits—it is more personalised and it is not affected by 'Do Not Disturb' (DND) filters.

The SMSLib API, developed by Thanasis Delenikas, is an open source library, available for Java and Microsoft .NET platforms. Through this API, applications can send and receive messages using almost any GSM modem. It's simple to implement and can interface with most of the USB GSM dongles available in the market.

Download

You have two options to use SMSLib in your projects. The first one is to download the source code of SMSLib v 3.5.x from <https://github.com/smslib/smslib-v3> and build it locally. If you choose this, follow the instructions available at <http://smslib.org/doc/installation/>. The second option is to download the compiled JAR file from <http://goo.gl/wx5oLZ> and import it into your NetBeans project using the steps mentioned previously.

Implementation

Since SMSLib uses the Java Communication API, you'll have to download and install the Java Comm API before using it. The following instructions will help you in downloading and installing the Java Comm API:

- Download the JavaComm v2 (for Windows 32-bit systems) from <http://smslib.org/download/> and extract the zip file.
- Copy `Comm.jar` to:
 - `%JAVA_HOME%\lib` (e.g., `C:\Program Files (x86)\Java\jdk1.8.0_11\lib`)
 - `%JAVA_HOME%\jre\lib\ext` (e.g., `C:\Program Files (x86)\Java\jdk1.8.0_11\jre\lib\ext`)
- Copy `win32com.dll` to:
 - `%JAVA_HOME%\bin` (e.g., `C:\Program Files (x86)\Java\jdk1.8.0_11\bin`)
 - `%JAVA_HOME%\jre\bin` (e.g., `C:\Program Files (x86)\Java\jdk1.8.0_11\jre\bin`)
 - `%windir%\System32` (e.g., `C:\Windows\System32`)
- Copy `javax.comm.properties` to:
 - `%JAVA_HOME%\lib` (e.g., `C:\Program Files (x86)\Java\jdk1.8.0_11\lib`)

- `%JAVA_HOME%\jre\lib` (e.g., `C:\Program Files (x86)\Java\jdk1.8.0_11\jre\lib`)



Note: The 'Program Files' (x86) folder exists only on 64-bit Windows machines.

Unfortunately, the JavaComm API is not available for 64-bit JDK; so in case you're using the 64-bit JDK, you'll have to use the RXTXComm API, which is an alternative to the JavaComm API and has a 64-bit version. The following instructions will help you in downloading and installing the RXTXComm API:

- Download the RXTXComm 64-bit version from <http://goo.gl/wx5oLZ> and extract the zip file.
- Copy `RXTXcomm.jar` to:
 - `%JAVA_HOME%\lib` (e.g., `C:\Program Files\Java\jdk1.8.0_11\lib`)
 - `%JAVA_HOME%\jre\lib\ext` (e.g., `C:\Program Files\Java\jdk1.8.0_11\jre\lib\ext`)
- Copy `rxtxSerial.dll` and `rxtxParallel.dll` to:
 - `%JAVA_HOME%\bin` (e.g., `C:\Program Files (x86)\Java\jdk1.8.0_11\bin`)
 - `%JAVA_HOME%\jre\bin` (e.g., `C:\Program Files (x86)\Java\jdk1.8.0_11\jre\bin`)
 - `%windir%\System32` (e.g., `C:\Windows\System32`).

Sample code

In the sample code snippets shown below:

- Replace the 'Gateway Name' with 'GSM modem gateway' (usually it's the manufacturer's name).
 - Replace the COM port with the COM port on which your device is registered. You can get the list of registered COM ports from `Computer > Properties > Device Manager > Ports (COM & LPT)`. Try each port, if multiple ports are listed there.
 - Change the baud rate according to your device.
- Sample code to send an SMS is:

```
SerialModemGateway smsGateway = new
SerialModemGateway("Huawei", "COM8", 9600, "", "");
smsGateway.setInbound(true);
smsGateway.setOutbound(true);

try {
    Service.getInstance().addGateway(smsGateway);
    Service.getInstance().startService();

    /*Insert the recipient phone number here*/

    OutboundMessage msg = new OutboundMessage("+91<Insert Phone
    Number here>", "Test Message");

    Service.getInstance().
    sendMessage(msg);

    Service.getInstance().stopService();
}
```